

SAMBA — agent-supported visual interactive control for distributed team building and empowerment

N Mehandjiev and B R Odgers

Using computers to co-ordinate work is meant to improve the efficiency of distributed teams, but has the potential to decrease effectiveness by fragmenting work into seemingly unrelated tasks. It can also replace team-based communication activities such as workload negotiation, which will make workers feel isolated and not in control of their work. If the workforce is demotivated, customer service quality and employee job satisfaction will suffer.

This paper shows how workload negotiation and informal team-building activities can be integrated with an operational support system for workload control and job scheduling. A proof-of-concept demonstrator, called SAMBA, has been built. The demonstrator uses agent-based negotiation strategies and allows workers and team managers to control the system in a visual interactive fashion. This type of system could be used to enable workers to set work preferences, trade jobs, share knowledge as well as build informal alliances to help each other with their work. Managers will be able to review and control local business rules and scheduling preferences.

The approach presented in SAMBA can be generalised to support team-based collaboration in environments where team members have different perspectives on the collaboration process and artefact, e.g. call centre staff could control call routing preferences, on-job training scheduling and so on.

1. Introduction

Computer-performed work co-ordination such as workload distribution and job scheduling can demotivate workers and thus decrease effectiveness in physically distributed teams. It has been shown [1] to fragment work into seemingly unrelated work tasks, and to impede *ad hoc* collaboration. It also removes workload negotiation and other team-based communication activities between workers, who start to feel isolated and not in control of their work. The demotivational effects of these factors can decrease customer service quality and employee job satisfaction, and thus negate the efficiency improvements brought about by the co-ordination technology.

This paper shows how workload negotiation and informal team-building activities can be supported by work co-ordination systems. This new approach uses agent-based negotiation and communication, and provides facilities for workers and team managers to control their work. Workers can set workload preferences such as task difficulty levels, and build informal alliances to help each other with their requests. Managers can review and control local business rules and scheduling preferences.

This approach is instantiated in a demonstration system called SAMBA (Software Agents for Mediating Business Activities). SAMBA is built using a distributed agent-based

architecture and visual direct-manipulation interface for control of work. It supports multiple viewpoints of work at individual as well as at group level.

The next sections review the requirements for an effective co-ordination support system for distributed teams. This paper also discusses how existing co-ordination systems support these requirements. The new approach is then presented, and it is shown how it can support work control and team building via distributed negotiation processes. This is followed by a description of why a visual direct-manipulation language should be used as an interface to work co-ordination systems. The architecture behind SAMBA is then presented. Discussing ways of generalising the SAMBA approach concludes this paper.

2. Requirements for computer support of distributed teamwork

Properly designed co-ordination systems can enrich the work environment and improve the effectiveness and motivation of workers. Unfortunately, far too often the opposite is witnessed, where technology decreases effectiveness and motivation by reducing variety of work and the autonomy of workers. The high turnover rate for call centre staff is often associated with the work being controlled, not

by workers, but rather by computerised scripts. A recent survey of customer service teams highlighted a perceived need for better control over the type of tasks and their level of difficulty, a need for workers to be able to sign themselves on or off, etc.

Issues of work control are clearly linked to the existing reward systems and to the collaborative culture in a team. Strategies for resolving conflicting demands and for balancing demands against expected workload should be developed, and managers at the appropriate organisational levels should be able to exercise control over these strategies. These strategies should be supplemented by measures to maximise the level of trust and collaborative culture in a team. Such factors are often ignored when introducing new technology. Indeed, electronic collaboration tools that replace face-to-face meetings have been shown to break down trust between the members of a distributed team [2]. Teams that use such tools have consistently shown mistrust and unwillingness to sacrifice personal short-term gains for maximised group benefit. This is an example where electronic collaboration technology has impeded collaboration.

Team collaboration and learning can also suffer if a badly designed co-ordination system is introduced. Work is fragmented into seemingly unrelated work pieces, and social interaction and problem-solving processes at the workplace are replaced by impersonal and strictly enforced commands regarding work. For example, the Trouble Ticketing System (TTS), which was introduced by a USA telephone company to act as a scheduling, work routing and record-keeping system, replaced the existing problem-solving conversations between repair technicians with indirect one-way messages, presuming that this would improve efficiency [1]. The result was the exact opposite, where the previously effective problem-solving network of people was replaced by a slow and ineffective 'relay race'. In this race, messages disembodied of context were handled in unpredictable ways by a disjointed array of workers. People did not know who was working on the same problem, and operated in isolation from their team. The community of practice was broken, with no one to train novices, since training was not an activity accounted for by the system. At the end the technicians found ways to contact each other and used the system to only record their work activities.

This case study shows that, to be successful, co-ordination technology has to recognise and support the way people collaborate to get work done. It has to support the webs of relationships existing between team members, where problems are discovered and resolved, knowledge is shared and work is done. This is especially important for physically distributed teams since in this context it is difficult for people to bypass technology and to establish conversations outside it. Indeed, after the introduction of

computer co-ordinated distributed working, repair technicians needed 'to have a chat', and started calling the control team very frequently for minor issues. Requirements that must be addressed by a co-ordination support system to achieve a good level of trust and collaborative culture can be summarised as follows.

- Workers should be able to control their working environment.
- Workers should individually and collectively be able to control the order and manner in which they perform tasks (within limits), etc.
- Managers should be able to specify and modify conflict resolution strategies and business-scheduling rules to balance expected workload with worker preferences.
- The co-ordination software should not just automate the co-ordination of work, it should also 'informate' workers about the process of work. For example, why was a request for overtime rejected, why is a piece of work considered urgent, who has worked on this task before, what is the business process in the context of which a particular activity takes place?
- Team-building activities should be supported by enabling workers to build a list of 'friends' to help each other out.
- Team learning and knowledge sharing activities should be supported by identifying people who work on the same task or in the same locality, and providing information to 'the expert' on a given issue.

The following section will look at the extent to which existing systems support this list of requirements.

3. Existing work control systems

Most work control systems implement a business process by co-ordinating a set of activities that comprise a model of that business process. Existing workflow systems use oversimplified PetriNet-like models of work and rigidly enforce these models. Because of this dictatorial style they are often called 'naziware' [3]. Freeflow [3] addresses this by providing flexible support for workers' activities rather than enforcing the process model. It lacks, however, facilities for work negotiation and team-building required for effective support of distributed team-work. It also lacks means for managerial influence and oversight. Indeed, this is a weakness in most workflow systems, each using a single 'correct' process model controlled by a single person. Collaboration while planning a business process is only supported by the research system Regatta VPL [4], and its commercial offshoot TeamWARE Flow. They allow business professionals to define and change 'their' section

of the process (their perspective) by manipulating a visual representation. However, both systems fail to support control and transparency across different perspectives.

Differences on perspectives and goals of different actors in a business process are catered for by agent-based process co-ordination systems. ADEPT [5] co-ordinates work through intelligent software agents which are engaged in constant negotiation processes to achieve their goals. Agent-based systems like ADEPT would appear to be suitable for building co-ordination systems for distributed teams because:

- they can provide a framework for dynamic changes and multiple concerns,
- they provide a level of abstraction that can help workers and managers to understand and control the system.

To achieve effective control by non-programmers such as workers and managers, appropriate visual interaction and work modelling techniques have to be devised on top of the agent infrastructure. The focus on visual interaction for multi-perspective co-ordination is what distinguishes the work described here from other agent-based co-ordination systems. The next two sections give an overview of this approach, and show how it used a highly interactive visual interface to enable non-programmers to control software.

4. The SAMBA approach

To address the requirements for effective work co-ordination software in the context of distributed teams, it is proposed that an agent-based framework is used. In this framework, each worker is paired with a software agent

called 'helper', as is the team manager. There is a central 'intermediary' agent for each team, as shown in Fig 1. This agent participates in the team co-ordination, and interfaces with BT operational support systems (BT OSS), such as billing and work scheduling systems. Information relevant to SAMBA is passed from the BT OSS to the intermediary. The intermediary uses this information along with predefined business logic to aid workers in their collaboration for business activities. When the workers formulate a mutually beneficial agreement the intermediary passes the result back to the BT OSS for use in the next evaluation cycle.

5. Work control

A worker can control their work environment by interacting with their helper. Control can either be exercised at run time (for example by selecting the next task) or at the planning stage (for example by specifying task difficulty preferences for the next day). This paper focuses on work control at the planning stage. Once workers specify their preferences, the Helper will negotiate these preferences with the team through a central 'intermediary' agent.

The intermediary employs a set of business rules and constraints to balance requests against expected workload for the team. The team manager can develop and change the set of business rules within the intermediary. This is done by the same mechanism as workers' preferences. The manager will modify the set of rules, and the changes will then be submitted to the intermediary, who will check if they are compatible with central control policies. Details of how workers and managers interact in this process are described later.

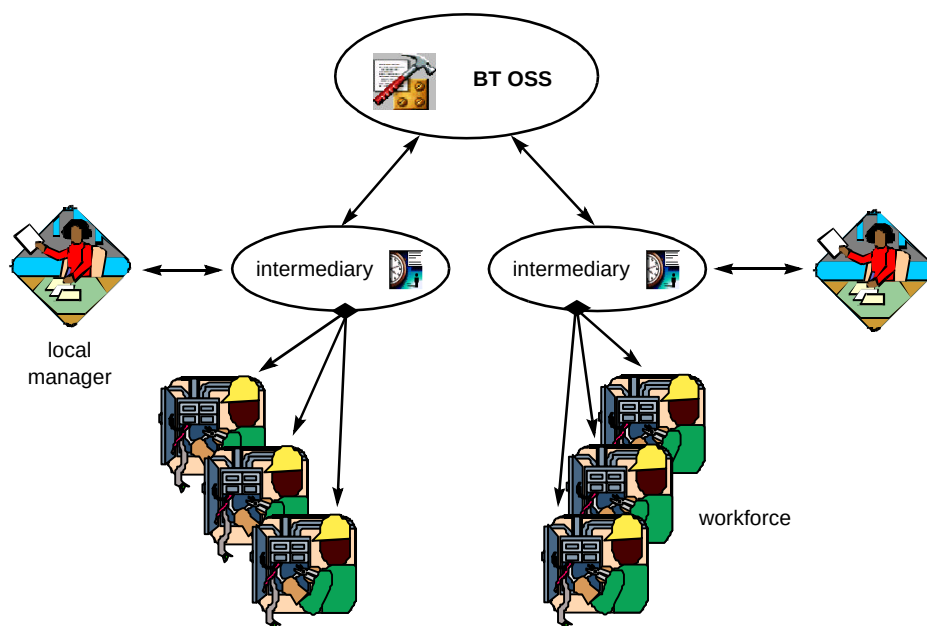


Fig 1 The intermediary agent is central for co-ordinating work in every team.

The same mechanism can be expanded to cover multiple levels of authority, and to allow different people to modify different aspects of the system.

6. Teamwork support

The framework provides support for team collaboration by notifying all helpers of the results of all requests, and the reasons for any rejection. This comparatively simple notification strategy enables a variety of negotiation and collaboration activities when combined with the intelligent behaviour of the helpers. This is illustrated in the following simple example.

John requests a day off. The intermediary checks his request against a business rule that for each worker who is taking a day off two team members need to take overtime. At the moment the system has only granted one request for overtime, and hence it will reject John's request for a day off work. It will broadcast the rejection together with the reasons for it to all members of the team.

- John's helper detects that his request has been rejected. The reason for the rejection is shown to John, and he is asked if the request should be re-submitted later in the day. The answer is 'yes', and this is remembered by the helper.
- Pat's helper detects that John's request for a day off has been rejected, and, since John is on Pat's 'friends' list, the Helper will ask Pat if she wants to help John out by taking overtime. Pat agrees, and the intermediary approves her request.
- John's helper receives the notification that Pat's request for overtime has been approved. The chances of John's original request being approved have improved, and John's helper will automatically re-submit the request for a day off. That is now approved as well.
- Jim is the team manager; his helper is tuned to notify him of any unresolved issues or repeatedly unsuccessful requests. Jim can then consider the situation, he may decide either to override the intermediary's decision on a one-off basis, or to change the set of business rules if this will improve them in the long run.

Using the framework of notifications between intelligent agents, work co-ordination applications can be built. These not only allow control at multiple levels of authority, but also inform workers of what is happening, why it is happening, and how they can help other members of their team. Thus support for team building and social interaction processes in a distributed work team can be achieved.

7. Visual interaction for work control

If a non-programmer is to control the behaviour and functionality of a software system effectively, the interface should provide a highly responsive style of interaction [6], in a manner similar to a spreadsheet. The system should provide immediate feedback regarding any effects of user actions, without going through a 'compile' step. Nardi [7] argues that, apart from immediate feedback, the system should communicate with the user in a language that is both task-oriented and visual. The power of such a language is its ability to externalise users' mental models of the control task [8].

Control at different levels of responsibility will therefore require different visual representations. For example there will be substantial differences between the visual interface for a worker, which can be constructed as a form with a limited set of work preferences, and the visual language employed by the manager to control business rules and conflict resolution policies. This is similar to the layered approach to visual software control and visual programming advocated by Repenning and Ioannidou [9]. They use programming competency as the factor to determine the type of representations at each layer, this paper proposes that the representations in a task-specific language will be determined by the type of task performed by people at different organisational levels.

At higher levels of managerial control the complexity and variety of control tasks will increase to an extent where a single representation will not be sufficient. A person will have to interact with several different representations depending on the type of control task they perform. A manager, for example, will need one type of representation to control business rules and another type of representation to control the team composition and preferences regarding type of work.

Facilities for integrating different representations into a unified control language with a high level of responsiveness are an important feature of this approach to building work co-ordination software.

8. SAMBA

To test this new approach to co-ordination of distributed teams, SAMBA — a proof-of-concept prototype system — has been built. SAMBA deals with the domain of job scheduling for customer service teams. In the prototype, workers can request days off, overtime, and specify preferred task difficulty. They do this through a single representation, but the system is designed so that other representations can be added. Also, different representations can be used for different types of hardware, for example a worker can use a mobile phone, a palm-top computer or PC (see Fig 2) to interface with the system.



Fig 2 User interface for PC user.

The team manager can control scheduling rules and constraints, and monitor the current team composition in terms of preferred task type, task difficulty, overtime and days off. In the prototype, the manager uses three different representations, which are tightly integrated together so that changes on one representation are immediately shown on the screen, and so are notifications from the intermediary (see Fig 3).



Fig 3 Intermediary information.

9. The architecture behind SAMBA

The system keeps users informed of the outcome of their and others' requests following the scenarios described earlier. It specifies reasons for rejection and suggests actions to help others. The architecture behind SAMBA can be described as follows. The system supports two main

modes of communication — a mode of synchronous visual interaction between users and their helpers, and asynchronous agent-based message passing and negotiation between the helpers and the Intermediary. This is reflected by the architecture presented in Fig 4.

The far left of Fig 4 shows how users of the system (workers or managers) will interact with a set of control windows specialised for the purposes of different control tasks. The user interactions with each window will be translated to commands for changing work co-ordination preferences, which can be understood by the helper. The translation for each control window will be performed by a separate module called 'translator' ('T' in Fig 4). Different pairs (control window, translator) can be plugged in and out of the 'bus', to provide different representations and different levels of control.

The 'helper' is a software agent which provides support for an individual user of the system, and maintains a local 'model' of their work and team preferences, including a list of 'friends'. The manager's helper will have a different functionality than the other helpers to provide a higher level of control.

Each helper will communicate with the 'intermediary' agent through a FIPA-compliant wrapper [10], denoted with 'S' in Fig 4. The agent communication bus (ACB) uses ACL (agent communication language) as a standard language for communication between the helpers and the intermediary.

The intermediary will maintain a 'confirmed model' of the team preferences. When a new request is received, it will be merged with the confirmed model to create a 'proposed model'. The reasoning part of the intermediary (denoted by 'ES' in Fig 4) will test whether the proposed model is a viable one in the context of the expected workload. If it is viable, the new request can be granted, otherwise it has to be rejected. If a request is granted, the proposed model becomes the confirmed model. If a request is rejected, the reason for rejection is sent to all helpers in the system to enable the behaviour described in the teamwork support section.

10. Conclusions and future work

The purpose of providing the framework described here is to support collaboration in distributed teams, where team members have different perspectives on the collaboration process and artefact, and a negotiation process is needed to balance personal perspectives against business constraints. The requirement for software being controlled by non-programmers determines the need for visually representing and managing this multiplicity of perspectives.

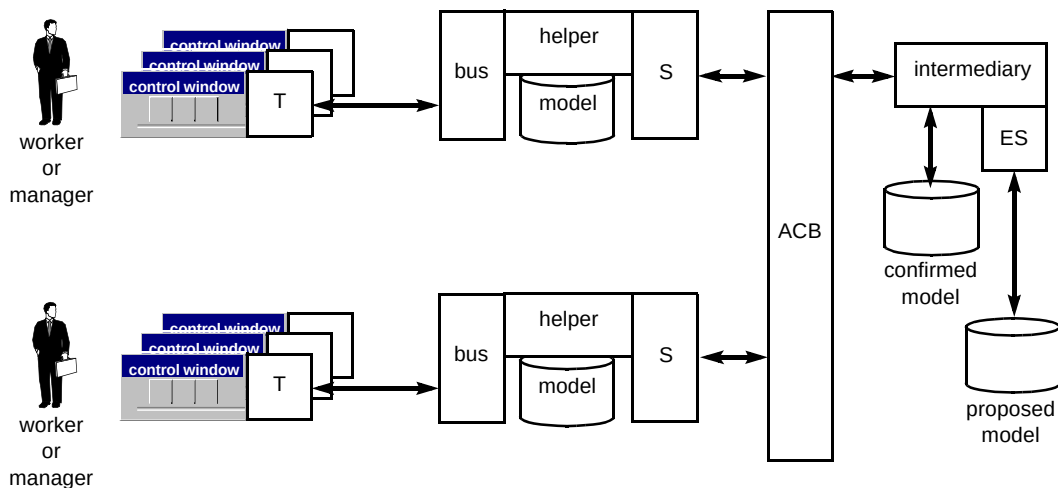


Fig 4 The architecture used to build SAMBA.

The approach presented here can be generalised in several directions. The architecture is capable of supporting different interfaces, so that for example touch-tone telephones can be used by customers to negotiate service times.

As differentiation between the different levels and types of control is done at the level of the local helper, managers at different levels of authority may therefore be connected to the system, e.g. higher levels of management may control scheduling policies and rules.

The general framework behind SAMBA can also be applied for domains different from job scheduling for customer service teams. One domain, which is considered quite appropriate for the next stage of development, is the domain of call centres, e.g. the system can enable call centre staff to control call routing preferences and participate in scheduling their on-job training.

References

- 1 Sachs P: 'Transforming work: collaboration, learning, and design', *Communications of the ACM*, **38**, No 9, pp 36—44 (September 1995).
- 2 Rocco E: 'Trust breaks down in electronic contexts but can be repaired by some initial face-to-face contact', *Proceedings of CHI 98* (April 1998).
- 3 Dourish P, Holmes J, MacLean A, Marquardsen P and Zbyslaw A: 'Freeflow: mediating between representation and action in workflow systems', in *ACM Computer Supported Cooperative Work*, ACM (November 1996).
- 4 Swenson K et al: 'A business process environment supporting collaborative planning', *Journal of Collaborative Computing*, **1**, No 1, pp 15—34 (March 1994).
- 5 Jennings N R, Faratin P, Johnson M J, O'Brien P, and Wiegand M E: 'Using intelligent agents to manage business processes', in *Proceedings of the First International Conference on The Practical Application of Intelligent Agents and Multi-Agent Technology (PAAM96)*, London, pp 345—360 (1996).
- 6 Burnett M M, Baker M J and van Zee P: 'Scaling up visual programming languages', *IEEE Computer*, **28**, No 3, pp 45 (March 1995).
- 7 Nardi B A: 'A small matter of programming: perspectives on end user computing', Massachusetts Institute of Technology (1993).
- 8 Mehandjiev N: 'User Enhanceability for Information Systems through Visual Languages', PhD thesis, Department of Computer Science, University of Hull, England (1997).
- 9 Repenning A and Ioannidou A: 'Behavior processors: layers between end-users and Java virtual machines', in *Proceedings of VL'97, IEEE* (September 1997).
- 10 FIPA: 'FIPA 98 Specifications', — <http://www.fipa.org/>



business modelling.

Dr Nikolay Mehandjiev is a Lecturer in the Department of Computation at UMIST, UK. He graduated in Computer Science from the Sofia Technical University in 1990, and then completed his PhD in 'User Enhanceability for Information Systems through Visual Programming' at the University of Hull, UK. His research interests are focused on systems development models, languages and tools in the context of rapidly changing business. He has active research projects in the areas of visual languages for end user development, agent-based architectures, flexible information systems and multi-perspective



Before joining BT in October 1995, Brian Odgers had gained a BSc, MSc and PhD at Queen's University, Belfast. Since then he has been working in the field of intelligent business systems, considering collaborative agents over a distributed environment. Recently he has been concerned with agent-based visualisation and monitoring of these systems. He now leads a team that is conducting research into intelligent process management and technologies. He is a technical advisor on the European funded FACTS project, which takes a 'do and discover' approach to evaluating agent standards bodies such as FIPA. In recent years, he has been a key contributor to numerous papers and four patent applications.